

Gaussian Processes, regression

## Machine Learning with Gaussians

- Due to the fact that we can analytically integrate over multivariate Gaussians they are heavily used of machine learning
- Examples include multivariate modelling, Kalman filters and Gaussian Processes
- Despite being analytically tractable, they are very fiddly
- This is mainly because it is easy to specify means,  $\mu$ , and covariance,  $C$ , but Gaussians involve the inverse covariance matrix

$$\mathcal{N}(x \mid \mu, C) = \frac{e^{-\frac{1}{2}(x-\mu)^T C^{-1}(x-\mu)}}{\sqrt{\det(2\pi C)}}$$

## Moment Generating Function

- Again through completing the square we can show

$$M(a) = \int_{-\infty}^{\infty} e^{a^T x} \mathcal{N}(x \mid \mu, C) dx = e^{a^T \mu + \frac{1}{2} a^T C a}$$

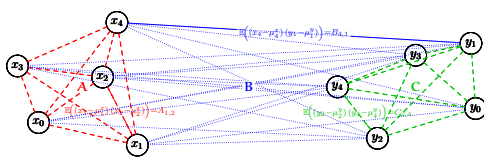
- The first moment is given by

$$\mathbb{E}[x] = \nabla_a M(a)|_{a=0} = (\mu + C a) e^{a^T \mu + \frac{1}{2} a^T C a} \Big|_{a=0} = \mu$$

- The second moment is

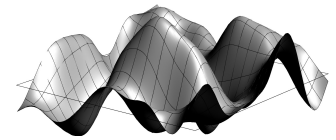
$$\mathbb{E}[x_i x_j] = \frac{\partial^2 M(a)}{\partial a_i \partial a_j} \Big|_{a=0} = \mu_i \mu_j + C_{ij}$$

## Multivariate Normals



$$\mathcal{N}\left(\begin{pmatrix} x \\ y \end{pmatrix} \mid \begin{pmatrix} \mu^x \\ \mu^y \end{pmatrix}, \begin{pmatrix} A & B \\ B^T & C \end{pmatrix}\right) = \frac{e^{-\frac{1}{2} \begin{pmatrix} x-\mu^x \\ y-\mu^y \end{pmatrix}^T \begin{pmatrix} A & B \\ B^T & C \end{pmatrix}^{-1} \begin{pmatrix} x-\mu^x \\ y-\mu^y \end{pmatrix}}}{\sqrt{\det(2\pi \begin{pmatrix} A & B \\ B^T & C \end{pmatrix})}}$$

- ML with Gaussians
- Gaussian Processes
- Bayesian Inference
- Hyper-parameters



## Completing the Square

- One of the key tools for performing Gaussian integrals is “completing the square”

$$\begin{aligned} -\frac{1}{2} x^T C^{-1} x + a^T x &= -\frac{1}{2} x^T C^{-1} x + \frac{1}{2} a^T x + \frac{1}{2} x^T a \\ &= -\frac{1}{2} x^T C^{-1} x + \frac{1}{2} a^T C C^{-1} x + \frac{1}{2} x^T C^{-1} C a \\ &= -\frac{1}{2} (x^T - a^T C) C^{-1} (x - C a) + \frac{1}{2} a^T C a \end{aligned}$$

- This gives us the Hubbard–Stratonovich transformation

$$\begin{aligned} \int_{-\infty}^{\infty} \frac{e^{-\frac{1}{2} x^T C^{-1} x + a^T x}}{\sqrt{|\det(2\pi C)|}} dx &= \int_{-\infty}^{\infty} \frac{e^{-\frac{1}{2} (x^T - a^T C) C^{-1} (x - C a) + \frac{1}{2} a^T C a}}{\sqrt{|\det(2\pi C)|}} dx \\ &= \int_{-\infty}^{\infty} \mathcal{N}(x \mid C a, C) e^{\frac{1}{2} a^T C a} dx = e^{\frac{1}{2} a^T C a} \end{aligned}$$

## Covariance

- The covariance is given by

$$\mathbb{E}[(x_i - \mu_i)(x_j - \mu_j)] = C_{ij}$$

$$\text{or } \mathbb{E}[(x - \mu)(x - \mu)^T] = C$$

- The cumulant generating function is given by

$$C(a) = \log(M(a)) = a^T \mu + \frac{1}{2} a^T C a$$

- The mean is equal to  $\nabla C(a)|_{a=0} = \mu$
- The covariance is given by  $\frac{\partial^2 C(a)}{\partial a_i \partial a_j} \Big|_{a=0} = C_{ij}$
- Normal distributions have no high cumulants (skewness, kurtosis)

## Marginalising Multivariate Normals

- I can define a moment generating function,

$$\begin{aligned} M(a, b) &= \int e^{(a^T, b^T) \begin{pmatrix} x \\ y \end{pmatrix}} \mathcal{N}\left(\begin{pmatrix} x \\ y \end{pmatrix} \mid \begin{pmatrix} \mu^x \\ \mu^y \end{pmatrix}, \begin{pmatrix} A & B \\ B^T & C \end{pmatrix}\right) dx dy \\ &= e^{\frac{1}{2} (a^T, b^T) \begin{pmatrix} A & B \\ B^T & C \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix}} = e^{\frac{1}{2} (a^T A a + b^T C b + 2 a^T B b)} \end{aligned}$$

- If I set  $b = 0$  this is equivalent to marginalising out  $y$
- But then we just get  $M(a, 0) = e^{\frac{1}{2} a^T A a}$
- This is the same as the moment generating function for  $\mathcal{N}(x \mid \mu^x, A)$

- An important and non-trivial result is

$$\int \mathcal{N}\left(\begin{pmatrix} x \\ y \end{pmatrix} \middle| \begin{pmatrix} \mu^x \\ \mu^y \end{pmatrix}, \begin{pmatrix} A & B \\ B^T & C \end{pmatrix}\right) dy = \mathcal{N}(x \mid \mu^x, A)$$

- You would think this is easy to prove by straightforward integration, but it ain't!
- It can be done, but it is a page of work!

## Gaussian Processes

- Gaussian processes (GPs) are a mathematically defined ensemble of functions
- They can be combined with Bayesian inference to give one of the most powerful regression techniques
- Although Bayesian they can be used in a black-box fashion due to the ubiquity of the prior
- Mathematically they are a bit complicated (because Gaussians involve the inverse of matrices which are a real pain to work with)
- In practice they aren't that difficult to use

## Priors on Functions

- We can think of a solution as a function  $f(x)$
- We can put a prior probability distribution,  $p(f)$ , on a function,  $f$ , that prefers smooth functions
- We can then compute a posterior probability distribution on functions given the data,  $p(f|\mathcal{D})$
- As a likelihood,  $p(y_i|f(x_i))$ , we use the probability of observing  $y_i$  given the true function value is  $f(x_i)$
- In general, this would be next to impossible to compute except in the special case where everything is Gaussian (normally distributed)

## Meaning of GP

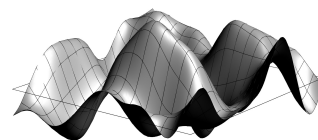
- To understand GP's we can discretise space,  $x$ , into a lattice of points  $\{x_i\}$
- Then (assuming  $m(x) = 0$ )

$$p(f|m, k) \propto \prod_i e^{-\frac{f_i^2 k^{-1}(x_i, x_i)}{2}} + f_i \sum_j k^{-1}(x_i, x_j) f_j$$

where  $f_i = f(x_i)$

- We see that the value of the function at each point is normally distributed with a mean that depends on functions at neighbouring points

1. ML with Gaussians
2. Gaussian Processes
3. Bayesian Inference
4. Hyper-parameters



## Regression

- In regression we try to fit a multi-dimensional function to our data
- (You can use Gaussian Processes for classification, e.g. by inferring the probabilities of being in a class, but we ignore this as regression is where GP excel)
- In regression we have some  $p$  dimensional feature vectors  $x_i$  and some target  $y_i \in \mathbb{R}$
- Our task is to fit a function through all the data points

## Gaussian Processes

- Gaussian Processes are probability distributions over functions
  - (Functions can be viewed as vectors in an infinite dimensional vector space)
  - In the Gaussian Process,  $\mathcal{GP}(m, k)$ , the probability of a function,  $f$ , is proportional
- $$p(f|m, k) \propto e^{-\frac{1}{2} \int (f(x) - m(x)) k^{-1}(x, y) (f(y) - m(y)) dx dy}$$
- The function  $m(x)$  is the mean  $\mathbb{E}[f(x)]$  (usually taken to be zero in most inference problems)

## Covariance function

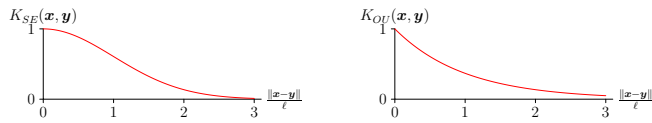
- $k(x, y)$  is a covariance function

$$\mathbb{E}[(f(x) - m(x))(f(y) - m(y))] = k(x, y)$$

- This is sometimes known as a **kernel**—it must be positive semi-definite (just like in SVMs)
- It is a free “parameter” that the user gets to choose (although we can learn its parameters too)
- If  $k(x, y)$  is a function of  $x - y$  it is “**stationary**”
- If  $k(x, y)$  is a function of  $\|x - y\|$  it is also “**isometric**”

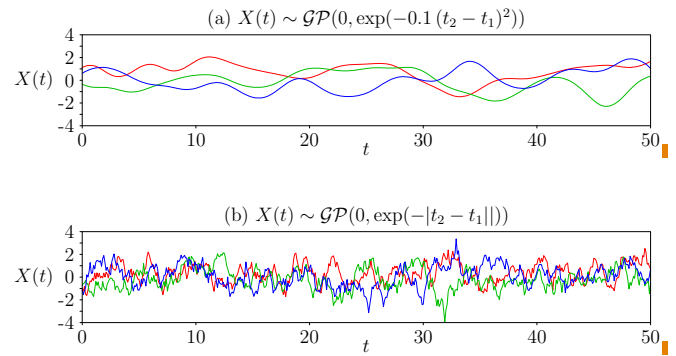
## Popular Choices of GP Kernel Function

- Constant:  $k_C(\mathbf{x}, \mathbf{y}) = C$
- Gaussian noise:  $k_{GN}(\mathbf{x}, \mathbf{y}) = \sigma^2 \delta_{\mathbf{x}, \mathbf{y}}$
- Squared exponential:  $k_{SE}(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{y}\|^2}{2\ell^2}\right)$
- Ornstein–Uhlenbeck:  $k_{OU}(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{y}\|}{\ell}\right)$



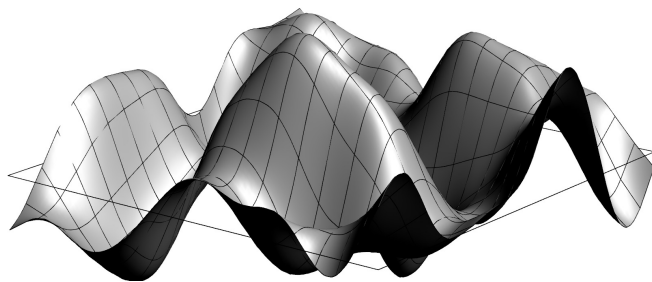
Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

## Gaussian Process Worlds



Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

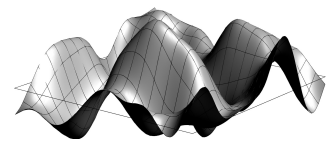
## 2-D Gaussian Processes



Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

## Outline

1. ML with Gaussians
2. Gaussian Processes
3. Bayesian Inference
4. Hyper-parameters



Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

## Observed Gaussian Processes

- Given some data points  $\mathcal{D} = ((\mathbf{x}_i, y_i) | i = 1, \dots, m)$  the likelihood (assuming Gaussian error are independence of the data point) is given by

$$p(\mathcal{D}|f) = \prod_{i=1}^m \mathcal{N}(y_i | f(\mathbf{x}_i), \sigma^2)$$

- Using a Gaussian Process prior we can compute a posterior using Bayes's rule
- The posterior is a Gaussian Process with a shifted mean and variance depending on the data-points
- This direct Bayesian derivation gives the answer involving the inverse matrix of the correlation function,  $k^{-1}(\mathbf{x}, \mathbf{y})$ —this is a pain to work with

Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

## Conditional Probability

- To compute the posterior  $p(f_* | \mathbf{y})$  we use

$$p(f_* | \mathbf{y}) = \frac{p(f_*, \mathbf{y})}{p(\mathbf{y})}$$

- where  $p(\mathbf{y}) = \int p(f_*, \mathbf{y}) df_*$
- Because all integrals are Gaussian we can compute the integral to obtain

$$p(f_* | \mathbf{y}) = \mathcal{N}\left(f_* \left| \mathbf{k}_*^T (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y}, k - \mathbf{k}_*^T (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{k}_* \right.\right)$$

- Looks complicated, but numerically easy to evaluate

Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

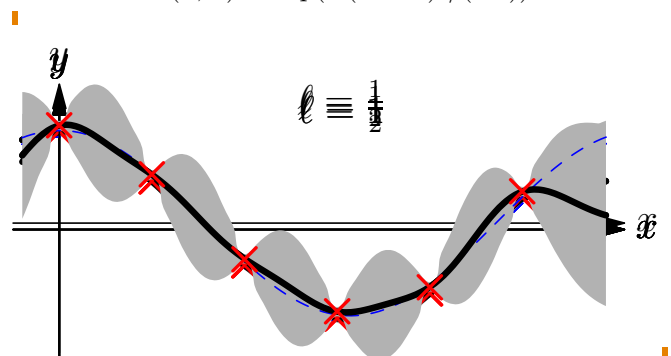
## Alternative Derivation

- Denoting the target values as a vector  $\mathbf{y}$  with elements  $y_i$
- Denoting the matrices of covariances between data points as  $\mathbf{K}$  with elements  $k(\mathbf{x}_i, \mathbf{x}_j)$
- Denoting the covariance between the data points and a particular position,  $\mathbf{x}_*$  as  $\mathbf{k}_*$  with elements  $k(\mathbf{x}_i, \mathbf{x}_*)$
- Denoting the variance a point  $\mathbf{x}_*$  as  $k_* = k(\mathbf{x}_*, \mathbf{x}_*)$
- Then the distribution of function values at points at  $\mathbf{x}_i$  and  $\mathbf{x}_*$  is

$$p(\mathbf{y}, f_*) = \mathcal{N}\left(\begin{pmatrix} \mathbf{y} \\ f_* \end{pmatrix} \left| \begin{pmatrix} \mathbf{K} + \sigma^2 \mathbf{I} & \mathbf{k}_* \\ \mathbf{k}_*^T & k_* \end{pmatrix} \right.\right)$$

Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

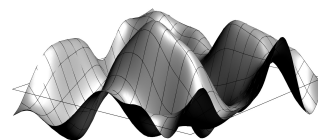
$$K(\mathbf{x}, \mathbf{x}') = \exp(-(x - x')^2 / (2\ell^2))$$



Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

- I've shown a 1-D regression example because it is easy to visualise
- This might be used with a time series
- The much more typical situation in machine learning is for  $\mathbf{x}$  to have many features so we are doing multi-dimensional regression
- Gaussian process inference were first used in spatial problems where it was known as **kriging**
- It was re-invented by the machine learning community who call it Gaussian Processes (GP)

1. ML with Gaussians
2. Gaussian Processes
3. Bayesian Inference
4. **Hyper-parameters**



## Choosing the Correct Covariance Function

- Choosing the correct covariance function is critical
- Most covariance functions include a continuous **hyper-parameter** (e.g. the correlation length  $\ell$ ) that we have to choose correctly
- This is typical of many Bayesian problems where we have some set of hyper-parameters,  $\phi$ , describing the model
- These are different to the normal parameters we learn (e.g. weights  $\mathbf{w}$  or in GP the functions  $f(\mathbf{x})$ )
- In Bayesian inference we learn the posterior for these normal parameters

$$p(f|\mathcal{D}, \phi) = \frac{p(\mathcal{D}|f, \phi)p(f|\phi)}{p(\mathcal{D}|\phi)}$$

## Maximum-Likelihood-II

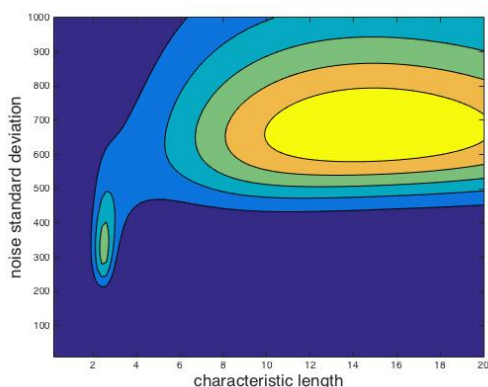
- The integral

$$p(f|\mathcal{D}) = \int p(f|\mathcal{D}, \phi)p(\phi|\mathcal{D})d\phi$$

usually can't be computed analytically and we have to use Monte Carlo methods (see later lecture)

- An alternative is to use the most likely hyper-parameter
- We can find this by using gradient search of  $p(\mathcal{D}|\phi)$
- This is sometimes referred to as ML-II
- Normally even this can be difficult, but for GP its not too difficult

## Example (slightly pathological)



## Evidence Framework

- The normalisation factor,  $p(\mathcal{D}|\phi)$  is known as the **marginal likelihood** or **evidence**

$$p(\mathcal{D}|\phi) = \int p(\mathcal{D}|f, \phi)p(f|\phi)df$$

- We can perform a Bayesian calculation at a second level by putting a prior on  $\phi$

$$p(\phi|\mathcal{D}) = \frac{p(\mathcal{D}|\phi)p(\phi)}{p(\mathcal{D})}$$

- From this we can now marginalise out the hyper-parameters

$$p(f|\mathcal{D}) = \int p(f|\mathcal{D}, \phi)p(\phi|\mathcal{D})d\phi$$

## Evidence for GP

- For GP the (log)-evidence can be computed in closed form

$$\log(p(\mathcal{D}|\phi)) = -\frac{1}{2}\mathbf{y}^T(\mathbf{K} + \sigma^2\mathbf{I})\mathbf{y} - \frac{1}{2}\log(|\mathbf{K} + \sigma^2\mathbf{I}|) - \frac{m}{2}\log(2\pi)$$

- ★ First term measures goodness of fit
- ★ Second term measure complexity of model
- ★ Last term is a common normalisation constant

- Can efficiently compute derivatives and find best parameters
- Could overfit!

## Conclusions

- Gaussian processes are very powerful for regression (and classification?)
- Because all calculations involve Gaussian integrals we can compute everything in closed form
- (Actually its a pain to do the mathematics because you end up working with inverse of matrices)
- Fairly generic (black-box) technique because the prior captures many continuity constraints
- We can use the evidence framework (probability of data) to do model selection and hyper-parameter optimisations